

Systeme d'allocation de portefeuille

Black-Litterman, IA, finance comportementale et interface Python

Livable	Application monofichier avec interface Tkinter
Cles API	Facultatives: fonctionnement local sans cle
Version	Mise a jour avec explication de l'IA et des pourcentages

Installation dans Visual Studio Code

1. Open the project folder

Dans VS Code: File > Open Folder..., puis selectionne le dossier du projet.

2. Open PowerShell

Clique Terminal > New Terminal. Si Bash apparait: fleche pres de + > Select Default Profile > PowerShell.

3. Create the environment

Cree l'environnement virtuel une seule fois.

```
python -m venv .venv
```

4. Install the dependencies

Utilise Python directement pour eviter le blocage Activate.ps1.

```
.\.venv\Scripts\python.exe -m pip install --upgrade pip
.\.venv\Scripts\python.exe -m pip install -r requirements.txt
```

5. Select the interpreter

Ctrl+Shift+P > Python: Select Interpreter > .venv\Scripts\python.exe.

6. Run the interface

Lance l'application depuis PowerShell.

```
.\.venv\Scripts\python.exe portfolio_bl_interface.py
```

L'IA est-elle reellement implementee ?

Oui, l'integration Anthropic Claude est presente dans `generate_views()`. Avec `ANTHROPIC_API_KEY`, Claude recoit les scores, le resume du marche et les headlines. Il produit pour chaque paire une position long/short, un rendement relatif attendu et une confiance.

Sans cle API, aucune IA generative externe n'est appelee. Le programme utilise `generate_fallback_views()`, un calcul local deterministe. L'interface indique la source: 'Anthropic', 'mode local sans cle API' ou 'mode local apres echec API'.

Le code ne demande pas actuellement a Claude de rediger une analyse narrative finale du portefeuille. L'IA sert a produire les vues relatives structurees qui alimentent Black-Litterman.

Comment le pourcentage relatif est-il calcule ?

En mode local, le programme compare les scores combines de deux actifs. Le meilleur score devient la position longue. Le rendement relatif est proportionnel a l'ecart absolu entre les scores.

```
difference = score_actif_1 - score_actif_2
magnitude = abs(difference)
expected_return = 0.12 * magnitude
borne finale: minimum 0.5%, maximum 15%
```

Exemple: score AAPL = 0.70 et score MSFT = -0.13. L'ecart vaut 0.83. Le calcul donne $0.12 \times 0.83 = 0.0996$, soit 9.96%. La vue signifie qu'AAPL devrait surperformer MSFT d'environ 9.96%. Elle ne signifie pas qu'AAPL augmentera nécessairement seul de 9.96%.

Passage dans Black-Litterman

Le rendement relatif devient une composante du vecteur Q. La ligne correspondante de P contient +1 pour l'actif long et -1 pour l'actif short. La confiance règle Omega: plus la confiance est forte, plus l'incertitude est faible et plus la vue influence le rendement posterior.

Limite importante: le coefficient 0.12 est une hypothèse de calibration. Ce n'est ni un modèle statistique entraîné, ni une garantie de hausse. Le résultat est une vue d'allocation soumise au risque de marche.

Paramètres principaux

Parametre	Exemple	Utilite
Actifs	AAPL, MSFT	Titres du portefeuille
Periode	2y	Historique Yahoo Finance
Seuil VIX	25	Poids comportemental renforce
Poids min/max	0.02 / 0.40	Bornes d'allocation
Taux sans risque	0.02	Ratio de Sharpe
Lambda / Tau	2.5 / 0.05	Parametres Black-Litterman

Code source complet

Le code ci-dessous correspond au fichier portfolio_bl_interface.py.

```

0001 """Application monofichier d'allocation Black-Litterman comportementale.
0002
0003 Python 3.11+. Les cles NEWS_API_KEY et ANTHROPIC_API_KEY sont facultatives.
0004 Sans cles, le sentiment est neutre et les vues relatives sont generees
0005 localement a partir des scores quantitatifs.
0006 """
0007
0008 from __future__ import annotations
0009
0010 import json
0011 import os
0012 import re
0013 import threading
0014 import traceback
0015 from datetime import datetime, timedelta, timezone
0016 from itertools import combinations
0017 from pathlib import Path
0018 from typing import Any, Callable
0019
0020 import matplotlib.pyplot as plt
0021 import numpy as np
0022 import pandas as pd
0023 import requests
0024 import tkinter as tk
0025 import yfinance as yf
0026 from matplotlib.backends.backend_tkagg import FigureCanvasTkAgg
0027 from pypfopt import EfficientFrontier
0028 from tkinter import messagebox, ttk
0029 from vaderSentiment.vaderSentiment import SentimentIntensityAnalyzer
0030
0031
0032 NEWS_API_URL = "https://newsapi.org/v2/everything"
0033
0034
0035 # -----
0036 # Collecte des donnees
0037 # -----
0038
0039
0040 def fetch_price_history(ticker: str, period: str) -> pd.DataFrame:
0041     """Telecharge les prix ajustes et calcule les rendements logarithmiques."""
0042     try:
0043         raw = yf.download(
0044             ticker,
0045             period=period,
0046             auto_adjust=False,
0047             progress=False,
0048             threads=False,
0049         )
0050     except Exception as exc:
0051         raise RuntimeError(f"Echec du telechargement pour {ticker}: {exc}") from exc
0052     if raw.empty:
0053         raise ValueError(f"Aucune donnee de prix pour {ticker}.")
0054     if isinstance(raw.columns, pd.MultiIndex):
0055         raw.columns = raw.columns.get_level_values(0)
0056     price_column = "Adj Close" if "Adj Close" in raw.columns else "Close"
0057     prices = pd.to_numeric(raw[price_column], errors="coerce").dropna()
0058     frame = pd.DataFrame(index=prices.index)
0059     frame["price"] = prices
0060     frame["log_return"] = np.log(frame["price"] / frame["price"].shift(1))
0061     return frame
0062
0063
0064 def fetch_news_sentiment(
0065     ticker: str,
0066     days: int = 7,
0067     api_key: str | None = None,
0068 ) -> tuple[float, list[str]]:
0069     """Calcule le sentiment VADER des titres NewsAPI, ou retourne 0 sans cle."""
0070     key = api_key or os.getenv("NEWS_API_KEY")
0071     if not key:
0072         return 0.0, []
0073     params = {
0074         "q": ticker,
0075         "from": (datetime.now(timezone.utc) - timedelta(days=days)).date().isoformat(),
0076         "language": "en",
0077         "sortBy": "publishedAt",
0078         "pageSize": 50,
0079         "apiKey": key,
0080     }
0081     try:
0082         response = requests.get(NEWS_API_URL, params=params, timeout=15)
0083         response.raise_for_status()
0084         articles = response.json().get("articles", [])
0085         headlines = [

```

```

0086         article["title"].strip()
0087         for article in articles
0088             if article.get("title") and article["title"] != "[Removed]"
0089             ]
0090         if not headlines:
0091             return 0.0, []
0092         analyzer = SentimentIntensityAnalyzer()
0093         scores = [analyzer.polarity_scores(title)["compound"] for title in headlines]
0094         return float(np.clip(np.mean(scores), -1.0, 1.0)), headlines
0095     except (requests.RequestException, ValueError, KeyError, TypeError):
0096         return 0.0, []
0097
0098
0099 def fetch_positioning_metrics(ticker: str) -> tuple[float, float | None]:
0100     """Recupere le put/call ratio et le nombre de titres vendus a decouvert."""
0101     put_call_ratio = np.nan
0102     short_interest: float | None = None
0103     try:
0104         asset = yf.Ticker(ticker)
0105         expirations = list(asset.options)
0106         if expirations:
0107             chain = asset.option_chain(expirations[0])
0108             call_oi = pd.to_numeric(
0109                 chain.calls.get("openInterest"), errors="coerce"
0110             ).sum()
0111             put_oi = pd.to_numeric(
0112                 chain.puts.get("openInterest"), errors="coerce"
0113             ).sum()
0114             if call_oi > 0:
0115                 put_call_ratio = float(put_oi / call_oi)
0116             value = (asset.info or {}).get("sharesShort")
0117             short_interest = float(value) if value is not None else None
0118     except Exception:
0119         pass
0120     return put_call_ratio, short_interest
0121
0122
0123 def collect_market_data(
0124     tickers: list[str],
0125     period: str,
0126     log: Callable[[str], None] = print,
0127 ) -> tuple[dict[str, pd.DataFrame], dict[str, list[str]]]:
0128     """Collecte les donnees de chaque actif sans interrompre tout le portefeuille."""
0129     datasets: dict[str, pd.DataFrame] = {}
0130     headlines: dict[str, list[str]] = {}
0131     for ticker in tickers:
0132         log(f"Telechargement de {ticker}...")
0133         try:
0134             frame = fetch_price_history(ticker, period)
0135             sentiment, ticker_headlines = fetch_news_sentiment(ticker)
0136             put_call_ratio, short_interest = fetch_positioning_metrics(ticker)
0137             frame["sentiment_score"] = sentiment
0138             frame["put_call_ratio"] = put_call_ratio
0139             frame.attrs["short_interest"] = short_interest
0140             datasets[ticker] = frame
0141             headlines[ticker] = ticker_headlines
0142             log(
0143                 f"{ticker}: {len(frame)} observations, "
0144                 f"prix={frame['price'].iloc[-1]:.2f}, sentiment={sentiment:+.2f}"
0145             )
0146         except (RuntimeError, ValueError) as exc:
0147             log(f"{ticker} ignore: {exc}")
0148     if len(datasets) < 2:
0149         raise RuntimeError("Au moins deux actifs valides sont necessaires.")
0150     return datasets, headlines
0151
0152
0153 def fetch_vix_level(default: float = 20.0) -> float:
0154     """Retourne le dernier VIX, ou une valeur neutre en cas d'echec."""
0155     try:
0156         data = yf.download(
0157             "^VIX", period="5d", auto_adjust=False, progress=False, threads=False
0158         )
0159         if isinstance(data.columns, pd.MultiIndex):
0160             data.columns = data.columns.get_level_values(0)
0161             close = pd.to_numeric(data["Close"], errors="coerce").dropna()
0162             return float(close.iloc[-1]) if not close.empty else default
0163     except Exception:
0164         return default
0165
0166
0167 # -----
0168 # Moteur de scoring
0169 # -----
0170
0171
0172 def bounded_zscore(value: float, series: pd.Series) -> float:
0173     """Transforme un z-score en signal borne entre -1 et +1."""
0174     clean = pd.to_numeric(series, errors="coerce").dropna()
0175     std = float(clean.std(ddof=1))

```

```

0176     if clean.empty or not np.isfinite(std) or std < 1e-12:
0177         return 0.0
0178     return float(np.tanh(((value - float(clean.mean())) / std) / 2.0))
0179
0180
0181 def calculate_fundamental_score(frame: pd.DataFrame) -> float:
0182     """Combine momentum 3 mois, volatilité et z-score du dernier rendement."""
0183     prices = frame["price"].dropna()
0184     returns = frame["log_return"].dropna()
0185     if len(prices) < 64 or len(returns) < 20:
0186         return 0.0
0187     momentum = float(np.tanh((prices.iloc[-1] / prices.iloc[-64] - 1.0) / 0.15))
0188     volatility = float(returns.tail(63).std(ddof=1) * np.sqrt(252))
0189     volatility_signal = float(-np.tanh((volatility - 0.25) / 0.20))
0190     return_z = bounded_zscore(float(returns.iloc[-1]), returns.tail(252))
0191     return float(np.clip(0.50 * momentum + 0.25 * volatility_signal + 0.25 * return_z, -1, 1))
0192
0193
0194 def calculate_behavioral_score(frame: pd.DataFrame) -> float:
0195     """Combine sentiment, sur-reaction a 20 jours et regime de prix."""
0196     prices = frame["price"].dropna()
0197     sentiment = float(frame["sentiment_score"].dropna().iloc[-1])
0198     if len(prices) < 63:
0199         return float(np.clip(sentiment, -1, 1))
0200     overreaction = -bounded_zscore(float(prices.iloc[-1]), prices.tail(20))
0201     momentum_20d = float(prices.iloc[-1] / prices.iloc[-21] - 1.0)
0202     momentum_3m = float(prices.iloc[-1] / prices.iloc[-63] - 1.0)
0203     agreement = np.sign(momentum_20d) * np.sign(momentum_3m)
0204     regime = float(np.tanh(momentum_20d / 0.08))
0205     if agreement <= 0:
0206         regime = -regime
0207     return float(np.clip(0.45 * sentiment + 0.35 * overreaction + 0.20 * regime, -1, 1))
0208
0209
0210 def score_assets(
0211     market_data: dict[str, pd.DataFrame],
0212     vix_level: float,
0213     vix_threshold: float,
0214 ) -> tuple[dict[str, float], dict[str, dict[str, float]]]:
0215     """Fusionne les scores avec davantage de comportemental en marche stressée."""
0216     behavioral_weight = 0.65 if vix_level > vix_threshold else 0.40
0217     scores: dict[str, float] = {}
0218     details: dict[str, dict[str, float]] = {}
0219     for ticker, frame in market_data.items():
0220         fundamental = calculate_fundamental_score(frame)
0221         behavioral = calculate_behavioral_score(frame)
0222         combined = (1 - behavioral_weight) * fundamental + behavioral_weight * behavioral
0223         scores[ticker] = float(np.clip(combined, -1, 1))
0224         details[ticker] = {
0225             "fundamental": fundamental,
0226             "behavioral": behavioral,
0227             "combined": scores[ticker],
0228         }
0229     return scores, details
0230
0231
0232 # -----
0233 # Generation des vues relatives
0234 # -----
0235
0236
0237 def build_market_snapshot(
0238     market_data: dict[str, pd.DataFrame],
0239 ) -> dict[str, dict[str, float]]:
0240     """Produit un resume compact du marche pour le generateur de vues."""
0241     snapshot: dict[str, dict[str, float]] = {}
0242     for ticker, frame in market_data.items():
0243         prices = frame["price"].dropna()
0244         returns = frame["log_return"].dropna()
0245         put_call = frame["put_call_ratio"].iloc[-1]
0246         snapshot[ticker] = {
0247             "last_price": float(prices.iloc[-1]),
0248             "return_20d": float(prices.iloc[-1] / prices.iloc[-21] - 1.0),
0249             "annual_volatility": float(returns.tail(252).std(ddof=1) * np.sqrt(252)),
0250             "sentiment": float(frame["sentiment_score"].iloc[-1]),
0251             "put_call_ratio": float(put_call) if pd.notna(put_call) else 1.0,
0252         }
0253     return snapshot
0254
0255
0256 def generate_fallback_views(scores: dict[str, float]) -> list[dict[str, Any]]:
0257     """Genere une vue pour chaque paire uniquement a partir des scores."""
0258     views: list[dict[str, Any]] = []
0259     for first, second in combinations(scores, 2):
0260         difference = scores[first] - scores[second]
0261         long_asset, short_asset = (
0262             (first, second) if difference >= 0 else (second, first)
0263         )
0264         magnitude = abs(difference)
0265         views.append(

```

```

0266         {
0267             "long": long_asset,
0268             "short": short_asset,
0269             "expected_return": float(np.clip(0.12 * magnitude, 0.005, 0.15)),
0270             "confidence": float(np.clip(0.35 + 0.50 * magnitude, 0.35, 0.90)),
0271         }
0272     )
0273     return views
0274
0275
0276 def build_llm_prompt(
0277     scores: dict[str, float],
0278     snapshot: dict[str, dict[str, float]],
0279     headlines: dict[str, list[str]],
0280 ) -> str:
0281     """Construit la demande JSON envoyee facultativement a Anthropic."""
0282     payload = {"scores": scores, "market": snapshot, "headlines": headlines}
0283     return (
0284         "Return JSON only: {\n\"views\": [{\n\"long\": \"TICKER\", \"\n\"short\": \"TICKER\", \"expected_return\": 0.05, \"confidence\": 0.7}]}.\n\"
0285         \"Create one relative view per unordered pair. expected_return must be \"
0286         \"between -0.30 and 0.30; confidence between 0.05 and 0.95.\n\"DATA:\n\"
0287         + json.dumps(payload, ensure_ascii=True, default=float)
0288     )
0289
0290
0291
0292 def parse_and_validate_views(text: str, assets: list[str]) -> list[dict[str, Any]]:
0293     """Extrait, valide et deduplique les vues JSON du modele."""
0294     cleaned = re.sub(r"```(?>json)?\s*|\\s*```$", "", text.strip(), flags=re.I)
0295     try:
0296         raw = json.loads(cleaned)
0297     except json.JSONDecodeError:
0298         match = re.search(r"```{.*\\}", cleaned, flags=re.S)
0299         if not match:
0300             raise ValueError("Aucun JSON detecte dans la reponse Anthropic.")
0301         raw = json.loads(match.group(0))
0302     valid_assets = set(assets)
0303     seen: set[frozenset[str]] = set()
0304     views: list[dict[str, Any]] = []
0305     for item in raw.get("views", []):
0306         long_asset = str(item.get("long", "")).upper()
0307         short_asset = str(item.get("short", "")).upper()
0308         pair = frozenset((long_asset, short_asset))
0309         if (
0310             long_asset not in valid_assets
0311             or short_asset not in valid_assets
0312             or long_asset == short_asset
0313             or pair in seen
0314         ):
0315             continue
0316         seen.add(pair)
0317         views.append(
0318             {
0319                 "long": long_asset,
0320                 "short": short_asset,
0321                 "expected_return": float(
0322                     np.clip(float(item.get("expected_return", 0)), -0.30, 0.30)
0323                 ),
0324                 "confidence": float(
0325                     np.clip(float(item.get("confidence", 0.5)), 0.05, 0.95)
0326                 ),
0327             }
0328         )
0329     if not views:
0330         raise ValueError("Aucune vue Anthropic valide.")
0331     return views
0332
0333
0334 def generate_views(
0335     scores: dict[str, float],
0336     snapshot: dict[str, dict[str, float]],
0337     headlines: dict[str, list[str]],
0338 ) -> tuple[list[dict[str, Any]], str]:
0339     """Utilise Anthropic si configure, sinon le generateur quantitatif local."""
0340     key = os.getenv("ANTHROPIC_API_KEY")
0341     if not key:
0342         return generate_fallback_views(scores), "mode local sans cle API"
0343     try:
0344         from anthropic import Anthropic
0345
0346         response = Anthropic(api_key=key).messages.create(
0347             model="claude-sonnet-4-20250514",
0348             max_tokens=4000,
0349             temperature=0.1,
0350             messages=[
0351                 {
0352                     "role": "user",
0353                     "content": build_llm_prompt(scores, snapshot, headlines),
0354                 }
0355             ],

```

```

0356         )
0357         text = "".join(
0358             block.text
0359             for block in response.content
0360             if getattr(block, "type", "") == "text"
0361         )
0362         return parse_and_validate_views(text, list(scores)), "Anthropic"
0363     except Exception:
0364         return generate_fallback_views(scores), "mode local apres echec API"
0365
0366
0367 # -----
0368 # Black-Litterman et optimisation
0369 # -----
0370
0371
0372 def black_litterman_returns(
0373     assets: list[str],
0374     covariance: pd.DataFrame,
0375     market_weights: pd.Series,
0376     views: list[dict[str, Any]],
0377     risk_aversion: float,
0378     tau: float,
0379 ) -> tuple[pd.Series, pd.Series]:
0380     """Calcule les rendements posterieurs Black-Litterman."""
0381     sigma = covariance.loc[assets, assets].to_numpy(dtype=float)
0382     weights = market_weights.reindex(assets).fillna(0.0)
0383     weights /= weights.sum()
0384     equilibrium = risk_aversion * sigma @ weights.to_numpy()
0385     index = {asset: position for position, asset in enumerate(assets)}
0386     rows, q_values, uncertainties = [], [], []
0387     for view in views:
0388         row = np.zeros(len(assets))
0389         row[index[view["long"]]] = 1.0
0390         row[index[view["short"]]] = -1.0
0391         confidence = float(np.clip(view["confidence"], 0.05, 0.95))
0392         variance = max(float(row @ (tau * sigma) @ row.T), 1e-8)
0393         rows.append(row)
0394         q_values.append(float(view["expected_return"]))
0395         uncertainties.append(max(variance * (1 - confidence) / confidence, 1e-8))
0396     p_matrix = np.vstack(rows)
0397     q_vector = np.asarray(q_values)
0398     omega = np.diag(uncertainties)
0399     prior_precision = np.linalg.pinv(tau * sigma)
0400     omega_precision = np.linalg.pinv(omega)
0401     posterior_precision = prior_precision + p_matrix.T @ omega_precision @ p_matrix
0402     posterior_rhs = (
0403         prior_precision @ equilibrium + p_matrix.T @ omega_precision @ q_vector
0404     )
0405     posterior = np.linalg.solve(posterior_precision, posterior_rhs)
0406     return (
0407         pd.Series(posterior, index=assets, name="mu_bl"),
0408         pd.Series(equilibrium, index=assets, name="pi"),
0409     )
0410
0411
0412 def optimize_portfolio(
0413     expected_returns: pd.Series,
0414     covariance: pd.DataFrame,
0415     min_weight: float,
0416     max_weight: float,
0417     risk_free_rate: float,
0418 ) -> tuple[pd.Series, tuple[float, float, float], str]:
0419     """Maximise le ratio de Sharpe avec les bornes de poids demandees."""
0420     assets = list(expected_returns.index)
0421     if min_weight * len(assets) > 1 or max_weight * len(assets) < 1:
0422         raise ValueError("Bornes de poids incompatibles avec le nombre d'actifs.")
0423     ef = EfficientFrontier(
0424         expected_returns,
0425         covariance.loc[assets, assets],
0426         weight_bounds=(min_weight, max_weight),
0427     )
0428     method = "Sharpe maximal"
0429     try:
0430         ef.max_sharpe(risk_free_rate=risk_free_rate)
0431     except (ValueError, ArithmeticError):
0432         ef = EfficientFrontier(
0433             expected_returns,
0434             covariance.loc[assets, assets],
0435             weight_bounds=(min_weight, max_weight),
0436         )
0437         ef.min_volatility()
0438         method = "volatilite minimale (secours)"
0439     weights = pd.Series(ef.clean_weights()).reindex(assets).fillna(0.0)
0440     performance = ef.portfolio_performance(risk_free_rate=risk_free_rate)
0441     return weights, performance, method
0442
0443
0444 def run_pipeline(
0445     tickers: list[str],

```

```

0446     period: str,
0447     vix_threshold: float,
0448     min_weight: float,
0449     max_weight: float,
0450     risk_free_rate: float,
0451     risk_aversion: float,
0452     tau: float,
0453     log: Callable[[str], None],
0454 ) -> dict[str, Any]:
0455     """Execute toute la chaine de calcul et retourne les resultats."""
0456     data, headlines = collect_market_data(tickers, period, log)
0457     assets = [ticker for ticker in tickers if ticker in data]
0458     vix = fetch_vix_level()
0459     scores, details = score_assets(data, vix, vix_threshold)
0460     log(f"\nVIX: {vix:.2f}")
0461     for ticker in assets:
0462         item = details[ticker]
0463         log(
0464             f"{ticker}: fondamental={item['fundamental']:+.3f}, "
0465             f"comportemental={item['behavioral']:+.3f}, "
0466             f"combine={item['combined']:+.3f}"
0467         )
0468     views, view_source = generate_views(scores, build_market_snapshot(data), headlines)
0469     log(f"\nVues: {view_source}")
0470     for view in views:
0471         log(
0472             f"Long {view['long']} / Short {view['short']}: "
0473             f"{view['expected_return']:+.2%}, confiance={view['confidence']:.0%}"
0474         )
0475     prices = pd.concat({ticker: data[ticker]['price'] for ticker in assets}, axis=1)
0476     prices = prices.sort_index().ffill().dropna()
0477     covariance = np.log(prices / prices.shift(1)).dropna().cov() * 252
0478     market_weights = pd.Series(1 / len(assets), index=assets)
0479     mu_bl, equilibrium = black_litterman_returns(
0480         assets,
0481         covariance,
0482         market_weights,
0483         views,
0484         risk_aversion,
0485         tau,
0486     )
0487     log("\nRendements Black-Litterman:")
0488     for ticker in assets:
0489         log(f"{ticker}: {mu_bl[ticker]:+.2%} (equilibre {equilibrium[ticker]:+.2%})")
0490     weights, performance, method = optimize_portfolio(
0491         mu_bl,
0492         covariance,
0493         min_weight,
0494         max_weight,
0495         risk_free_rate,
0496     )
0497     expected_return, volatility, sharpe = performance
0498     log("\nAllocation finale:")
0499     for ticker, weight in weights.sort_values(ascending=False).items():
0500         log(f"{ticker}: {weight:.2%}")
0501     log(
0502         f"\n[method] | rendement={expected_return:.2%} | "
0503         f"volatilite={volatility:.2%} | Sharpe={sharpe:.2f}"
0504     )
0505     return {
0506         "weights": weights,
0507         "expected_return": expected_return,
0508         "volatility": volatility,
0509         "sharpe": sharpe,
0510     }
0511
0512
0513 # -----
0514 # Interface graphique
0515 # -----
0516
0517
0518 class PortfolioApp(tk.Tk):
0519     """Fenetre principale de configuration et de restitution."""
0520
0521     def __init__(self) -> None:
0522         super().__init__()
0523         self.title("Allocation Black-Litterman comportementale")
0524         self.geometry("1120x760")
0525         self.minsize(920, 650)
0526         self.configure(bg="#eef2f7")
0527         self.variables = {
0528             "tickers": tk.StringVar(value="AAPL, NVDA, MSFT, GOOG, META"),
0529             "period": tk.StringVar(value="2y"),
0530             "vix_threshold": tk.StringVar(value="25"),
0531             "min_weight": tk.StringVar(value="0.02"),
0532             "max_weight": tk.StringVar(value="0.40"),
0533             "risk_free_rate": tk.StringVar(value="0.02"),
0534             "risk_aversion": tk.StringVar(value="2.5"),
0535             "tau": tk.StringVar(value="0.05"),

```



```

0536     }
0537     self._build_ui()
0538
0539     def _build_ui(self) -> None:
0540         """Construit les controles, le journal et la zone graphique."""
0541         style = ttk.Style(self)
0542         style.theme_use("clam")
0543         style.configure("Title.TLabel", font=("Segoe UI", 18, "bold"), foreground="#17324d")
0544         style.configure("Run.TButton", font=("Segoe UI", 10, "bold"))
0545
0546         header = ttk.Frame(self, padding=(18, 14))
0547         header.pack(fill="x")
0548         ttk.Label(
0549             header,
0550             text="Allocation Black-Litterman avec signaux comportementaux",
0551             style="Title.TLabel",
0552         ).pack(anchor="w")
0553         ttk.Label(
0554             header,
0555             text="Fonctionne sans cle API. Les donnees de prix proviennent de Yahoo Finance.",
0556         ).pack(anchor="w", pady=(4, 0))
0557
0558         body = ttk.Panedwindow(self, orient="horizontal")
0559         body.pack(fill="both", expand=True, padx=16, pady=(0, 16))
0560         left = ttk.Frame(body, padding=14)
0561         right = ttk.Frame(body, padding=10)
0562         body.add(left, weight=1)
0563         body.add(right, weight=2)
0564
0565         fields = [
0566             ("Actifs (separés par des virgules)", "tickers"),
0567             ("Periode Yahoo (1y, 2y, 5y...)", "period"),
0568             ("Seuil VIX", "vix_threshold"),
0569             ("Poids minimum", "min_weight"),
0570             ("Poids maximum", "max_weight"),
0571             ("Taux sans risque", "risk_free_rate"),
0572             ("Aversion au risque lambda", "risk_aversion"),
0573             ("Tau Black-Litterman", "tau"),
0574         ]
0575         for row, (label, key) in enumerate(fields):
0576             ttk.Label(left, text=label).grid(row=row * 2, column=0, sticky="w", pady=(5, 2))
0577             ttk.Entry(left, textvariable=self.variables[key], width=34).grid(
0578                 row=row * 2 + 1, column=0, sticky="ew", pady=(0, 4)
0579             )
0580         left.columnconfigure(0, weight=1)
0581         self.run_button = ttk.Button(
0582             left,
0583             text="Executer l'allocation",
0584             style="Run.TButton",
0585             command=self.start_run,
0586         )
0587         self.run_button.grid(row=17, column=0, sticky="ew", pady=(14, 5))
0588         ttk.Button(left, text="Effacer le journal", command=self.clear_log).grid(
0589             row=18, column=0, sticky="ew"
0590         )
0591
0592         notebook = ttk.Notebook(right)
0593         notebook.pack(fill="both", expand=True)
0594         log_frame = ttk.Frame(notebook)
0595         chart_frame = ttk.Frame(notebook)
0596         notebook.add(log_frame, text="Resultats")
0597         notebook.add(chart_frame, text="Graphique")
0598
0599         self.log_text = tk.Text(
0600             log_frame,
0601             wrap="word",
0602             font=("Consolas", 10),
0603             bg="#fbfcfe",
0604             fg="#1d2a36",
0605             padx=10,
0606             pady=10,
0607         )
0608         scrollbar = ttk.Scrollbar(log_frame, command=self.log_text.yview)
0609         self.log_text.configure(yscrollcommand=scrollbar.set)
0610         self.log_text.pack(side="left", fill="both", expand=True)
0611         scrollbar.pack(side="right", fill="y")
0612
0613         self.figure, self.axes = plt.subplots(1, 2, figsize=(8, 4.8))
0614         self.figure.patch.set_facecolor("#fbfcfe")
0615         self.canvas = FigureCanvasTkAgg(self.figure, master=chart_frame)
0616         self.canvas.get_tk_widget().pack(fill="both", expand=True)
0617         self._draw_empty_chart()
0618
0619     def _draw_empty_chart(self) -> None:
0620         """Affiche un etat initial dans la zone graphique."""
0621         for axis in self.axes:
0622             axis.clear()
0623             axis.axis("off")
0624         self.axes[0].text(
0625             0.5,

```

```

0626         0.5,
0627         "Le graphique apparaîtra\naprès le calcul.",
0628         ha="center",
0629         va="center",
0630         fontsize=13,
0631         color="#607080",
0632     )
0633     self.canvas.draw_idle()
0634
0635     def append_log(self, text: str) -> None:
0636         """Ajoute une ligne au journal depuis le thread principal."""
0637         self.after(0, self._append_log_now, text)
0638
0639     def _append_log_now(self, text: str) -> None:
0640         self.log_text.insert("end", text + "\n")
0641         self.log_text.see("end")
0642
0643     def clear_log(self) -> None:
0644         self.log_text.delete("1.0", "end")
0645
0646     def read_inputs(self) -> dict[str, Any]:
0647         """Valide et convertit les champs de l'interface."""
0648         tickers = [
0649             value.strip().upper()
0650             for value in self.variables["tickers"].get().split(",")
0651             if value.strip()
0652         ]
0653         tickers = list(dict.fromkeys(tickers))
0654         if len(tickers) < 2:
0655             raise ValueError("Saisissez au moins deux tickers.")
0656         values = {
0657             "tickers": tickers,
0658             "period": self.variables["period"].get().strip() or "2y",
0659             "vix_threshold": float(self.variables["vix_threshold"].get()),
0660             "min_weight": float(self.variables["min_weight"].get()),
0661             "max_weight": float(self.variables["max_weight"].get()),
0662             "risk_free_rate": float(self.variables["risk_free_rate"].get()),
0663             "risk_aversion": float(self.variables["risk_aversion"].get()),
0664             "tau": float(self.variables["tau"].get()),
0665         }
0666         if not 0 <= values["min_weight"] <= values["max_weight"] <= 1:
0667             raise ValueError("Les poids doivent vérifier 0 <= min <= max <= 1.")
0668         if values["tau"] <= 0 or values["risk_aversion"] <= 0:
0669             raise ValueError("Tau et lambda doivent être strictement positifs.")
0670         return values
0671
0672     def start_run(self) -> None:
0673         """Démarrage le pipeline dans un thread pour garder l'interface réactive."""
0674         try:
0675             inputs = self.read_inputs()
0676         except ValueError as exc:
0677             messagebox.showerror("Paramètres invalides", str(exc))
0678             return
0679         self.clear_log()
0680         self.append_log("Démarrage du calcul...\n")
0681         self.run_button.configure(state="disabled")
0682         threading.Thread(
0683             target=self._run_worker,
0684             args=(inputs,),
0685             daemon=True,
0686         ).start()
0687
0688     def _run_worker(self, inputs: dict[str, Any]) -> None:
0689         """Exécute le calcul et transmet le résultat à l'interface."""
0690         try:
0691             result = run_pipeline(log=self.append_log, **inputs)
0692             self.after(0, self.show_chart, result)
0693             self.append_log("\nCalcul termine.")
0694         except Exception as exc:
0695             self.append_log(f"\nERREUR: {exc}")
0696             self.append_log(traceback.format_exc())
0697             self.after(0, messagebox.showerror, "Echec du calcul", str(exc))
0698         finally:
0699             self.after(0, self.run_button.configure, {"state": "normal"})
0700
0701     def show_chart(self, result: dict[str, Any]) -> None:
0702         """Dessine et sauvegarde les graphiques d'allocation."""
0703         weights: pd.Series = result["weights"]
0704         for axis in self.axes:
0705             axis.clear()
0706             axis.axis("on")
0707         colors = plt.cm.Set2.colors[: len(weights)]
0708         self.axes[0].pie(
0709             weights.values,
0710             labels=weights.index,
0711             autopct="%1.1f%%",
0712             startangle=90,
0713             colors=colors,
0714         )
0715         self.axes[0].set_title("Repartition")

```

```
0716         weights.sort_values().plot.barh(ax=self.axes[1], color=colors)
0717         self.axes[1].set_title(f"Poids - Sharpe {result['sharpe']:.2f}")
0718         self.axes[1].set_xlabel("Poids")
0719         self.axes[1].xaxis.set_major_formatter(lambda value, _: f"{value:.0%}")
0720         self.figure.tight_layout()
0721         self.canvas.draw_idle()
0722         output = Path(__file__).with_name("portfolio_allocation_interface.png")
0723         self.figure.savefig(output, dpi=160, bbox_inches="tight")
0724         self.append_log(f"Graphique enregistré: {output.resolve()}")
0725
0726
0727 if __name__ == "__main__":
0728     PortfolioApp().mainloop()
```